# 2、程序、静态库、动态库、配置文件

# 嵌入式设备应用开发（程序、静态库、动态库、配置文件）

原创

嵌入式-老费 已于 2023-08-26 08:57:20 修改

阅读量355 收藏

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： linux

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

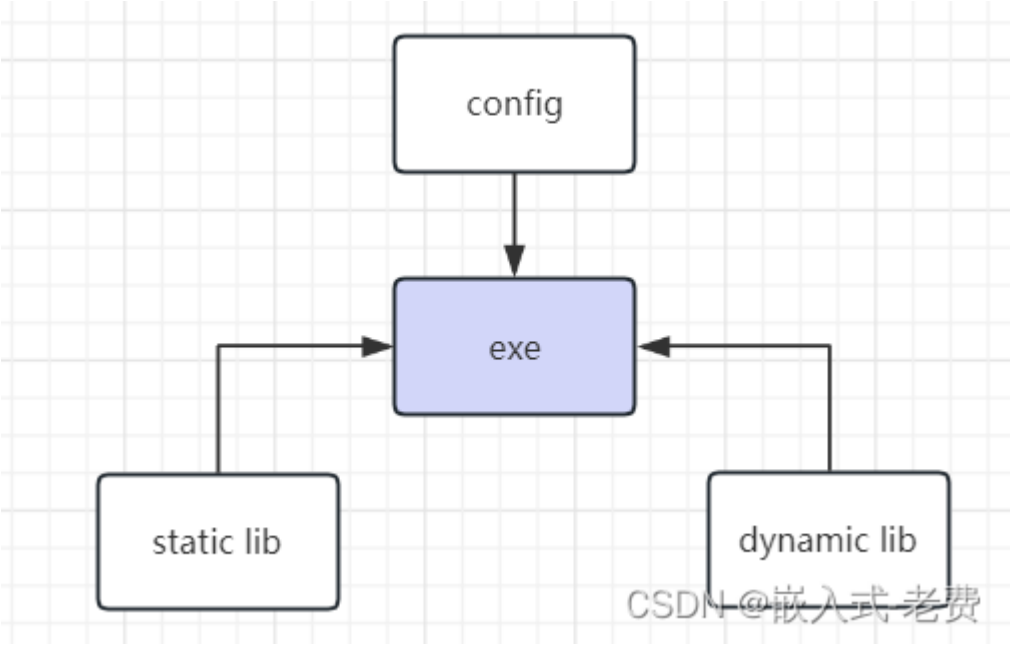
摘要 本文介绍了嵌入式设备应用开发的过程，包括程序、静态库、动态库和配置文件的编译与使用。针对不同类型的库，详细阐述了交叉编译和链接的步骤，强调了动态库在调试方面的优势以及配置文件对于程序灵活性和可扩展性的重要性。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

一个程序要想正确地在设备上运行起来，那么基本步骤就是，首先编写好程序代码，接着用交叉编译器编译出来，最后将这个程序拷贝到嵌入式设备上。然后，我们可以通过console控制台的方式将这个程序运行起来。这基本上就是嵌入式应用程序开发的步骤。

如果是稍微复杂一点的程序，那么过程就稍微多一点。首先要看，这个程序是否有关联的静态库需要编译；接着看是否有关联的动态库需要编译；最后一步就是查看一下和程序关联的配置文件。这中间缺少了哪一步，程序都是没有办法正确运行。当然这里没有提到设备接口，因为我们默认设备驱动都是ok的，没有其他的问题。




## 1、程序

所谓的程序，大家就可以看成是main函数所在的工程，假设有这样的代码demo.c，

```
#include <stdio.h>

int main(int argc, char* argv[])
```

```
{
    printf("demo in embed system.\n");
    return 0;
}
```

如果程序只是在x86上面运行，那么就比较简单，

```
gcc demo.c -g -o demo
```

但是我们的目的是想让程序可以在嵌入式设备运行的，所以这个时候编译器就不能选择gcc了，而应该根据我们自己的设备选择arm-linux-gnueabi-gcc这个编译器，

```
arm-linux-gnueabi-gcc demo.c -g -o demo
```

虽然生成的都是demo文件，但是我们可以用file命令看下，就知道哪个文件是x86生成的，哪个文件是arm生成的，

```
feixiaoxing@feixiaoxing-VirtualBox:~/Desktop$ file demo
demo: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.3, for GNU/Linux
3.2.0, BuildID[sha1]=a0ce85dfd33346b9e476cbc61ff7e0a9b47d8845, with
debug_info, not stripped
```

## 2、静态库

一般的程序都需要依赖很多的第三方库，有的是开源的，有的是闭源的，这都有可能。一般情况下，如果程序不是很复杂，所有的程序都编译成静态库，这是很方便的。静态库编译完成后，不需要拷贝到嵌入式设备上，因为相关的的代码已经被合并到exe文件里面去了，不再需要额外的拷贝动作了。假设有一个add.c这样的文件，

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}
```

编译的方法也很简单，主要是分成两步，第一步生成.o文件，第二步生成.a文件，

```
arm-linux-gnueabi-gcc -c add.c -o add.o
arm-linux-gnueabi-ar rc libadd.a add.o
```

libadd.a编译好了，如果需要把它加入到demo程序中，只需要用gcc编译的时候，把.a文件放到最后就好了，

```
arm-linux-gnueabi-gcc demo.c -g -o demo ./libadd.a
```

## 3、动态库

和静态库一样，动态也是使用比较多的一种方式。但是和静态库相比，动态库也有自己的优势。这主要是体现在debug调试方面。 **试想一下，如果静态库出现了问题，那么所有用到静态库的程序，都需要重新编译一遍。而这个时候如果用到的是动态库，那么我们只需要把动态库编译一下，然后把它拷贝到嵌入式开发板上就行了，这就是最大的区别。** 还是上面的add.c，如果编译成动态库，应该怎么编译呢，

```
arm-linux-gnueabi-gcc -shared -fPIC -o libadd.so add.c
```

同样，此时如果需要合并到demo.c也不复杂，

```
arm-linux-gnueabi-gcc demo.c -g -o demo ./libadd.so
```

看上去和静态库是一样的，但是大家注意，一定要把libadd.so拷贝到嵌入式开发板上，设置好环境变量，demo才能运行起来的。 **所谓的环境变量就是LD_LIBRARY_PATH，这一点一定要注意下的。**

## 4、配置文件

配置文件很多，有算法参数，有功能参数等等。我们简单分析下，简单的参数，我们可以通过命令行，在程序启动的时候直接就给它了。但是复杂一点的参数，比如哪些功能需要打开、是否需要打印调试、算法中的参数是否需要重新标定，这些都是可以用配置文件来完成的。

**试想一下，如果没有配置文件会怎么样？那就是我们对程序所作的任何一点修改，都需要重新编译、重新下载、重新测试** 。而这个修改，可能仅仅是某个数字、某个范围、某个参数发生了改变而已。如果有这样的配置文件，那么测试和标定的同学，就可以通过配置和修改后面的参数，独立进行后面的工作，不需要程序员的参与和介入了。对这个程序而言，它的应用范围也更广了，本身程序也更加灵活了。